In linguaggio ANSI C, siano dichiarate le seguenti variabili, e siano queste variabili debitamente inizializzate:

```
pthread_mutex_t mutex;
pthread_cond_var cond;
```

considerare poi la seguente porzione di codice, in cui condizione sia una espressione da valutare:

```
while (condizione) {
   pthread_cond_wait(&cond, &mutex);
}
fai_qualcosa();
```

La porzione di codice, sopra riportata, effettua busy-waiting?

No, la porzione di codice fornita non effettua busy-waiting.
Al contrario, utilizza la funzione pthread_cond_wait, che è una forma di attesa condizionale che sospende l'esecuzione del thread corrente fino a quando la condizione non è soddisfatta.

*** Una stessa struttura dati ha la stessa dimensione in tutti i processori? Perché?**

No, dipende dall'architettura della CPU (64 o 32 bit), la dimensione potrebbe variare anche in base al sistema operativo (dipende dalla strategia di allocazione usata).

*** In un programma in C dichiaro un vettore di elementi, in cui gli elementi sono un tipo di struttura dati struct A. Guardando come viene collocato in memoria quel vettore, può accadere che tra un elemento ed il successivo, dello stesso vettore, ci siano dei byte non occupati dalla struttura dati di tipo struct A? Perchè?**

Sì, è possibile che tra gli elementi di un vettore di struct ci siano dei byte non occupati, questo fenomeno è noto come "padding" o "allineamento di memoria". Il compilatore può inserire byte aggiuntivi tra i membri della struttura per garantire un corretto allineamento in memoria e per ottimizzarne l'accesso.

In linguaggio C, se un programma è costituito da più moduli, è possibile che una stessa funzione (cioè una funzione con stesso nome, stessi tipi di argomenti e stesso tipo di dato restituito) sia implementata in due diversi moduli senza che questo provochi un errore nella generazione del programma eseguibile? Perché?

Se hai la stessa funzione definita in due o più moduli, e stai cercando di creare un eseguibile che include entrambi questi moduli, riceverai un errore di collegamento a causa delle "multiple definition" della stessa funzione.

La seguente istruzione si trova nella funzione main di un modulo scritto in C.
ris = PRODOTTO (3 - 4 , SOMMA (2, 7));
Precedentemente, nello stesso file, le due macro sono state così definite:
#define PRODOTTO(X, Y) ((X)*(Y))
#define SOMMA(X , Y) ((X)+(Y))
Se quel modulo viene processato dal preprocessore del compilatore gcc, in che modo viene tradotta quella istruzione dal preprocessore?

Quando il modulo viene processato dal preprocessore del compilatore GCC, l'istruzione:

ris = PRODOTTO(3 - 4, SOMMA(2, 7));

verrà tradotta in:

ris = ((3 - 4) * ((2) + (7)));

Questo processo è il risultato della sostituzione delle macro PRODOTTO e SOMMA con le rispettive definizioni fornite.
Il preprocessore sostituisce ogni occorrenza delle macro nel codice sorgente con il corpo della macro corrispondente, applicando eventuali argomenti forniti.

Supponiamo di inserire le seguenti due istruzioni, in un modulo scritto in C, in una zona fuori da tutte le funzioni del modulo;
#define NUM 10
int vet[10];
Se quel modulo viene processato dal preprocessore del compilatore gcc, in che modo viene tradotta quella istruzione dal preprocessore?

Il preprocessore effettuerà semplicemente la sostituzione del testo, quindi ogni occorrenza della macro NUM verrà sostituita con il valore 10. Quindi il codice sarà uguale a quello dato in partenza.

Spiegare cosa si intende con Race Condition.

Una "Race Condition" è una situazione in cui il comportamento di un programma dipende dall'ordine di esecuzione delle operazioni concorrenti svolte da più thread. Si verifica quando almeno due thread accedono simultaneamente a una risorsa condivisa modificandola e non sono utilizzati meccanismi di sincronizzazione per garantire la mutua esclusione. La mancanza di sincronizzazione porta a risultati indesiderati.

Spiegare cosa si intende quando si dice che una sequenza di istruzioni viene eseguita in maniera "atomica".

Una sequenza di istruzioni viene definita "atomica" quando viene eseguita come un'operazione singola e indivisibile. In questo caso "atomica" significa che l'operazione viene eseguita senza interruzioni da parte di altri thread concorrenti.
Nessun'altra operazione che usa i dati condivisi con l'operazione atomica può cominciare prima che non sia terminata; quindi non può esistere interleaving tra le diverse operazioni.
Nel caso in cui fosse necessario interrompere un'operazione atomica, bisognerà in seguito farla ripartire da capo.

Un Task può avere più stack a sua disposizione?
Perché sì o perché no?

Si, in vari casi, ad esempio la creazione di diversi thread all'interno di un processo, ognuno con un proprio stack.
Inoltre, ogni processo ha a disposizione uno stack per ogni privilege ring della CPU.
Infine, alcuni sistemi mettono a disposizione diversi stack per la gestione di Interrupt ed Eccezioni.

Spiegare come dovrebbe funzionare l' "algoritmo ottimale" di sostituzione delle pagine.

È un algoritmo teorico che sostituisce la pagina in memoria che verrà utilizzata più tardi nel futuro. L'obiettivo è minimizzare il numero di sostituzioni di pagine, cercando di prevedere quali pagine non saranno utilizzate per un periodo di tempo più lungo.
L'algoritmo ottimale è considerato teorico perché richiede una conoscenza perfetta del futuro comportamento del programma, cosa che è impossibile da ottenere nella pratica.

* Spiegare come funziona l' algoritmo "Least Recently Used (LRU)" di sostituzione delle pagine

LRU ipotizza che se una pagina non è utilizzata di recente, è meno probabile che venga utilizzata di recente. Si usa una struttura dati per mantenere traccia delle pagine in memoria, ogni volta che una pagina viene usata, viene spostata in cima alla lista, in modo che la pagina in fondo sia quella che è stata usata meno di recente. Quando la memoria è piena la pagina infondo alla lista viene sostituita con quella meno recente ma comunque più recente di essa.

*** Consideriamo una finestra del working set di ampiezza 6, e consideriamo un programma che accede alle proprie pagine in questa sequenza, 3 5 7 2 8 5 6 5 4 5 6 7 1 5 6 4.**
Supponiamo ora che sia stata appena acceduta l'ultima pagina della sequenza indicata. Indicare qual è il working set.

L'ultima pagina della sequenza è il 4, consideriamo i 5 numeri prima di questa: il working set attuale è {1, 4, 5, 6, 7}, che rappresenta le ultime 5 pagine accedute prima dell'accesso alla pagina "4".

* Che cos'è il thrashing?

Il thrashing si verifica quando il sistema operativo è costantemente impegnato a sostituire le pagine in memoria.
Ciò si verifica ad esempio quando la memoria non è sufficiente per supportare l'esecuzione dei processi in corso. In questo caso la CPU viene impegnata per la sostituzione delle pagine, riducendo significativamente le prestazioni generali. Il thrashing può essere evitato aumentando la memoria o modificando l'algoritmo di sostituzione utilizzato oppure usando il paging della memoria.

Nell'ambito della paginazione per la memoria virtuale, cosa si intende con "allocazione dei frame ai processi".

È il processo di assegnazione di porzioni fisiche di memoria, chiamate frame, ai processi eseguiti dal sistema. La memoria fisica del sistema è divisa in blocchi di dimensioni fisse chiamati frame, mentre la memoria virtuale di ciascun processo è suddivisa in blocchi chiamati pagine. Il sistema operativo è responsabile di gestire la corrispondenza tra pagine di memoria virtuale e frame di memoria fisica.

Spiegare a cosa serve il procedimento di sostituzione delle pagine nell'ambito della gestione della memoria virtuale.

Serve a massimizzare l'utilizzo della memoria fisica, gestire le risorse limitate e minimizzare i tempi di accesso. L'algoritmo di sostituzione delle pagine determina quali pagine mantenere in memoria fisica e quali spostare su uno storage di massa, contribuendo a garantire l'efficienza e l'esecuzione di più processi contemporaneamente.

In che contesto si parla di "dirty bit" e a che cosa serve?

Il dirty bit è un flag utilizzato nella gestione della memoria virtuale, che indica se una pagina è stata modificata o meno. Se il bit è a 0 la pagina non è stata modificata, può quindi semplicemente essere rimossa dalla memoria, altrimenti se è a 1 la pagina è stata modificata e quindi è necessario prima copiarla sul disco per poterla rimuovere dalla memoria.

Spiegare cosa avviene allo scatenarsi di un page fault.

Un page fault è un'eccezione di tipo FAULT che si verifica quando il processore tenta di accedere a una pagina di memoria che non è attualmente caricata in memoria. Quando avviene un page fault il pager carica la pagina dal disco nella memoria fisica (swap-in) e la CPU riprende l'esecuzione.

* Spiegare cosa sono i processi "I/O bound" e quelli "CPU bound".

Processi I/O Bound (Input/Output Bound):

- Caratteristiche: Questi processi trascorrono la maggior parte del loro tempo eseguendo operazioni di input/output, ad esempio, lettura o scrittura su dispositivi di archiviazione (disco).
- Comportamento: Solitamente, questi processi sono più limitati dalla velocità di I/O rispetto alla potenza di elaborazione della CPU. La CPU attende spesso che l'I/O sia completato prima di poter eseguire ulteriori operazioni.
- Esempio: Un'applicazione che legge dati da un file invia richieste di rete o elabora flussi di dati in ingresso e in uscita può essere I/O bound.

Processi CPU Bound (CPU Bound):

- Caratteristiche: Questi processi trascorrono la maggior parte del loro tempo impegnati in operazioni di calcolo intensivo sulla CPU. Solitamente, richiedono più tempo di CPU rispetto all'I/O.
- Comportamento: Questi processi sono limitati dalla velocità di elaborazione della CPU e spesso utilizzano la CPU in modo intensivo per eseguire compiti computazionali complessi.
- Esempio: Un algoritmo di elaborazione di immagini, un calcolo scientifico intensivo o un'operazione di rendering grafico sono esempi di processi CPU bound.

* Spiegare la differenza tra "scheduling preemptive" e "scheduling non preemptive".

- Scheduling Preemptive: Il sistema operativo può interrompere un processo in esecuzione per assegnare la CPU a un altro processo.
- Scheduling Non Preemptive: Il sistema operativo non può interrompere un processo in esecuzione fino a quando non completa la sua esecuzione o si verifica un evento di blocco.

* L'algoritmo di scheduling FIFO soffre di starvation? Perché?

Sì, in certe situazioni. La ragione per cui può soffrire di starvation è perchè assegna la CPU in base all'ordine di arrivo dei processi. Se un processo con un tempo di esecuzione più lungo o infinito si trova all'inizio della coda e continuano ad arrivare nuovi processi, quelli successivi potrebbero non essere mai eseguiti fino a quando il processo iniziale non completa la sua esecuzione.

* Cos'è la formattazione fisica o formattazione di basso livello?

È il processo di preparazione di un dispositivo di archiviazione per l'uso nel sistema.
Questo processo coinvolge la creazione della struttura di base organizzata in settori, tracce e cilindri, stabilendo la struttura fisica che consente la scrittura e la lettura di dati.

Spiegare come avviene il bootstrap nei sistemi moderni.

Quando il pc viene acceso, la CPU inizia a leggere delle istruzioni in memoria ROM che corrispondono al bootstrap, queste istruzioni indicano alla CPU dove sono i dati del sistema operativo che vanno caricati in RAM, poi il programma indica alla CPU dove iniziare a leggere le istruzioni del sistema operativo che imposta il proprio livello di privilegio come massimo e prende il controllo della macchina.

* A cosa serve il programma fdisk?

fdisk è un comando Unix per gestire le partizioni di un disco rigido.
Serve per creare, modificare, eliminare le partizioni, visualizzarne le informazioni e impostare quale usare all'avvio.

* A cosa serve il programma ln?

ln è un comando Unix per creare collegamenti (link) tra file. Esistono due tipi di link supportati da ln: i link hard e i soft link.

* Devo implementare una delle politiche di scheduling del disco magnetico descritte a lezione, ma ancora non so quale. Che informazioni mi servono per potere implementare una qualunque di quelle politiche?

Non fatto

* Perché non parliamo di politiche di scheduling del disco quando abbiamo a che fare con dischi a stato solido?

Non fatto

*** Ipotizziamo che i cilindri di un disco magnetico siano indicizzati in maniera crescente dalla periferia verso il centro. Supponiamo che la testina di lettura di un disco si trovi sul cilindro di indice 10.**
Ipotizziamo che la testina, per arrivare alla posizione attuale, si sia spostata verso il centro del disco.
Supponiamo che l'elenco delle richieste pendenti sia la seguente, ordinata in ordine di tempo di arrivo delle richieste.
12, 15, 6, 3, 6, 11, 13, 17, 20, 1.
Se uso un algoritmo di scheduling Shortest Seek Time First (SSTF), quale sarà la sequenza temporale dei prossimi accessi a disco?

Non fatto

*** Nelle stesse ipotesi dell'esercizio precedente, quale sarà la sequenza temporale dei prossimi accessi a disco se invece uso una politica di scheduling del disco di tipo CLOOK?**

Non fatto

*** Spiegare la differenza tra Hard link e Soft link in ambito Linux**

Un link hard è un collegamento diretto a un inode all'interno del sistema.
Il soft link è un file che contiene il percorso del file originale.

Cos'è un inode?

È una struttura dati che contiene le informazioni del file necessarie per identificarlo e gestirlo, come le autorizzazioni, i timestamp, il tipo di file e i puntatori ai blocchi di dati del file.

Che struttura si utilizza, in ambiente Linux, per memorizzare i blocchi di un file?

I blocchi di un file vengono memorizzati utilizzando una struttura di dati chiamata inode.

In linguaggio ANSI C, sia dato il seguente programma:

```
#include <stdlib.h>
int main(void) {
char str[120] = "ciao";
   char *p;
   p = &str;
   return(0);
}
```

Il programma viene correttamente compilato e linkato. Procediamo ad eseguirlo.
Durante l'esecuzione, alla variabile p viene assegnato l'indirizzo di inizio della stringa str.
Dopo l'assegnamento, l'indirizzo contenuto nella variabile p specifica un indirizzo in memoria fisica oppure in memoria virtuale?

Specifica un indirizzo di memoria virtuale che in fase di esecuzione verranno mappati nei corrispondenti indirizzi di memoria fisica dal Memory Management Unit.

Definire cos'è un Sistema di Elaborazione dell'Informazione.

È un insieme di componenti interconnessi e coordinati progettati per acquisire, elaborare, memorizzare e distribuire informazioni.

Cosa accade durante il bootstrap? E dove si trovano le istruzioni che guidano il bootstrap?

Quando il pc viene acceso, la CPU inizia a leggere delle istruzioni in memoria ROM che corrispondono al bootstrap, queste istruzioni indicano alla CPU dove sono i dati del sistema operativo che vanno caricati in RAM, poi il programma indica alla CPU dove iniziare a leggere le istruzioni del sistema operativo che imposta il proprio livello di privilegio come massimo e prende il controllo della macchina.
Le istruzioni di bootstrap si trovano nel BIOS del computer, memorizzate in una piccola memoria non volatile chiamata ROM.

* Una stringa è collocata in memoria in modo diverso se la CPU è di tipo Big Endian invece che Little Endian?

Sì, la disposizione in memoria di una stringa può variare tra Big Endian e Little Endian.
Nel Big Endian i byte più significativi vengono collocati all'indirizzo di memoria più basso,
nel Little Endian i byte meno significativi vengono collocati all'indirizzo di memoria più basso.

* Dichiarare una struttura dati pacchettizzata.

```
struct Pacchetto {
   int numero;
   char carattere;
} attribute_((packed))STRUTTURA;
```

Sapendo che una CPU lavora "a 64 bit" possiamo dire qualcosa a proposito di qualche BUS del computer?

Con una CPU a 64 bit:
Bus di Dati: Trasferimenti di dati a 64 bit alla volta.
Bus di Indirizzi: Supporta indirizzi a 64 bit.
Bus di Sistema: Trasferimenti di dati a 64 bit.
Registri Interni: Registri larghi 64 bit.

* Cos'è e a cosa serve il DMA?

Il DMA è un meccanismo hardware dedicato al trasferimento dati che consente di ridurre al minimo il coinvolgimento della CPU.
Il trasferimento dei dati, dal disco alla memoria, effettuato dal DMA non coinvolge la CPU tranne che:
- Quando la CPU ordina al DMA lo spostamento del blocco dati.
- Quando il DMA avvisa la CPU dell'avvenuto trasferimento.

A cosa serve l'opzione -I del gcc?

L'opzione -I serve per indicare la posizione dei file header.

A cosa serve l'opzione -L del gcc?

L'opzione -L serve per indicare la posizione delle librerie (statiche o dinamiche).

Cosa sono i Livelli di Privilegio (Rings) della CPU?

I "Livelli di Privilegio" della CPU si riferiscono a una gerarchia di privilegi che vengono utilizzati nei processori per gestire l'esecuzione di istruzioni e l'accesso alle risorse del sistema .

Cos'è una system call?

Le system call sono servizi messi a disposizione dal sistema operativo che normalmente il processo non potrebbe eseguire e sono disponibili mediante chiamate ad interrupt.
Sono funzionalità utilizzate da linguaggi di alto livello.

Qual è il compito del loader?

Quando un programma viene lanciato, il loader (che fa parte del sistema operativo):

- crea i segmenti di memoria per il processo
- carica in memoria il codice eseguibile del programma
- carica in memoria la sezione Data
- crea lo stack utente per l'esecuzione delle funzioni del programma
- crea gli stack di sistema per l'esecuzione delle system calls che verranno invocate dal programma utente
- copia eventuali argomenti passati a riga di comando mettendoli a disposizione del main
- ordina alla CPU di eseguire la prima istruzione eseguibile corrispondente al main del programmala sezione di codice, la sezione di dati e le risorse del sistema operativo.

Indicare la differenza tra FAULT ed ABORT e dare un esempio per ciascuna delle due eccezioni.

FAULT:

- condizione anomala o errore che può essere gestito senza interrompere drasticamente l'esecuzione del programma.
- Esempio: "Page Fault", si verifica quando un programma cerca di accedere a una pagina di memoria che non è attualmente caricata in memoria fisica.

ABORT:

- Interruzione forzata dell'esecuzione del programma a causa di un errore critico o di una condizione irreversibile.
- Esempio: "Division by Zero" (divisione per zero), rappresenta una situazione in cui il risultato di un'operazione di divisione è indefinito e potrebbe portare a comportamenti imprevedibili. In questo caso, il sistema può decidere di interrompere immediatamente l'esecuzione del programma per evitare danni ulteriori.

Spiegare la differenza tra Interrupt Sincroni ed Interrupt Asincroni.

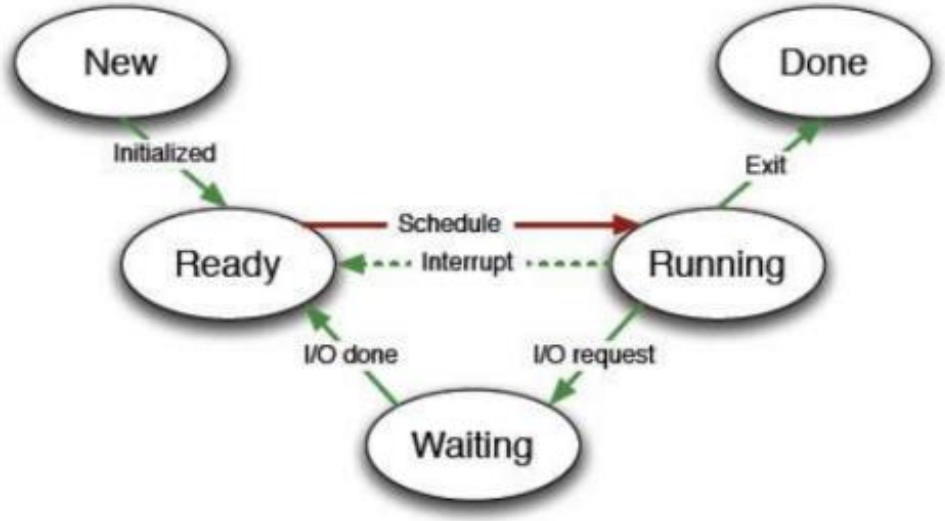
Interrupt Sincroni:

Generati come risposta a istruzioni specifiche del programma.
Causati da eventi interni al programma.
Esempio: Divisione per zero.

Interrupt Asincroni:

Generati da eventi esterni al flusso di esecuzione normale.
Causati da segnali hardware o periferiche esterne.
Esempio: Interruzione da parte di un dispositivo di input.

Disegnare il diagramma di stato dei processi, dal punto di vista dello scheduler.



Spiegare in che situazione un processo si trova in stato waiting.

Un processo si trova in stato di "waiting" quando attende l'attivazione di una risorsa o l'occurrence di un evento che gli consentirà di procedere con la sua esecuzione.

Nella bash, qual è la differenza tra variabili locali e variabili d'ambiente?

Una variabile locale è una variabile visibile e modificabile solo all'interno della shell corrente. Una variabile d'ambiente di una shell viene ereditata mediante copia anche dalle sue subshell.

Nella bash, qual è la differenza tra una variabile vuota e una variabile che non esiste. È possibile eliminare una variabile?

Una variabile vuota è una variabile che è stata creata ma non inizializzata mentre una variabile che non esiste non è ancora stata creata.
È possibile eliminarla con il comando unset nomevariabile.

Cos'è la funzione di Autocompletamento della bash

È una funzionalità che mette a disposizione bash per facilitare l'inserimento dei comandi.
Per utilizzarla basta scrivere le prime lettere di un comando e premere TAB.

Che cosa hanno in comune i Posix thread di uno stesso processo?

I POSIX thread di uno stesso processo condividono lo stesso spazio di indirizzamento, quindi i file descriptor, le variabili globali e le risorse del processo (dati globali e CPU).

* Se un processo possiede diversi posix thread, e quel processo genera un processo figlio, il processo figlio possiede dei thread?

Sì, il processo figlio eredita una copia identica dell'ambiente di lavoro del processo padre a partire dal momento della creazione. Il processo figlio eredita un solo thread dal genitore, cioè il thread che aveva chiamato la fork().

* Se un processo possiede diversi posix thread, e quel processo genera un processo figlio, i thread del processo figlio "vedono" le variabili globali del processo padre oppure no?

In generale si può dire che le variabili globali non sono condivise fra processo padre e processo figlio, tuttavia il sistema operativo in realtà condivide inizialmente le pagine, perché fork() implementa un meccanismo copy-on-write, il che significa che a condizione che nessuno dei processi modifichi le pagine, esse sono condivise.

Che cos'è errno?

errno è una variabile di errore che ogni thread può avere. Contiene un valore significativo solo se la chiamata è stata effettuata con un errore. Per utilizzare questa variabile dobbiamo includere l'header errno.h.

* Se un primo thread esegue una chiamata a funzione di libreria del C, e questa produce un errore e scrive il codice d'errore nella variabile errno, un altro thread può controllare il contenuto della variabile errno per conoscere il tipo di errore capitato nel primo thread?

No, la variabile errno è specifica per ogni thread.

Definire la Liveness.

La liveness indica il corretto funzionamento di un thread, in pratica la capacità di "vivere" fino al compimento delle operazioni senza entrare in stallo (Deadlock o Starvation).

Definire il Busy Waiting

Il busy waiting consiste in un loop infinito in cui un thread controlla continuamente una condizione finché non viene rispettata, causando spreco di risorse in quanto il thread continua a chiedere l'accesso ad una risorsa non disponibile finché non lo diventa.

Definire il Deadlock.

Due o più thread sono in Deadlock se entrambi aspettano indefinitamente un evento che può essere causato solo da uno dei thread bloccati.

Definire la Starvation.

La starvation si verifica quando un thread non riesce a ricevere le risorse di cui ha bisogno per completare il proprio lavoro, ad esempio se non ha mai priorità sufficiente per accedere ai dati in presenza di altri thread con priorità più alta.

Che cosa sono i processi Zombie?

I processi Zombie sono quei processi che, pur essendo terminati, continuano ad occupare memoria inutilmente. È per questo che il processo padre una volta che i processi figli terminano deve pensare a deallocare totalmente la memoria da loro occupata, ossia ad ucciderli, facendo una wait.

Descrivere concisamente qualche tipo di strumento, messo a disposizione dallo standard POSIX, per sincronizzare tra loro dei processi.

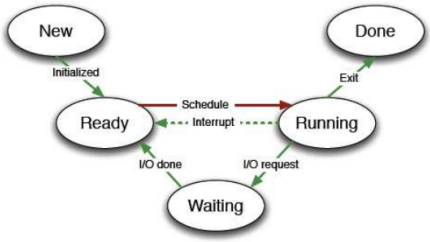
Mutex lock → blocca una risorsa finchè non viene rilasciata dalla unlock
Mutex unlock → sblocca una risorsa bloccata dalla lock
Cond wait, signal, broadcast

Spiegare che cosa è la rilocazione statica

È un'operazione svolta dal linker che modifica le istruzioni di salto cambiando l'operando che specifica l'indirizzo a cui saltare per specificare correttamente l'indirizzo in cui si trova l'istruzione da eseguire. La rilocazione statica riguarda tutti i salti assoluti.

Spiegare che cosa è il binding degli indirizzi

È l'associazione di indirizzi di memoria fisica ai dati e all'istruzione di un programma (indirizzi virtuali). Può avvenire:
- durante la compilazione, allora gli indirizzi rimarranno invariati ad ogni esecuzione e parliamo di codice assoluto
- durante il caricamento, cioè è il loader a aggiornare tutti i riferimenti coerentemente alla punto iniziale di caricamento (codice rilocabile)
- durante l'esecuzione, allora l'indirizzo di memoria effettivo viene calcolato durante l'esecuzione dalla MMU che prende in input un indirizzo logico e restituisce l'indirizzo fisico corrispondente.



TEORIA SISTEMI OPERATIVI YYY01

1. In linguaggio ANSI C, sia dato il seguente programma:

```
#include <stdlib.h>
int main (void) {
     char str[120] = "ciao";
     char *p;
     p = &str;
     return (0);
}
```

Il programma viene correttamente compilato e linkato. Procediamo ad eseguirlo. Durante l'esecuzione, alla variabile p viene assegnato l'indirizzo della stringa str.

Dopo l'assegnamento, l'indirizzo contenuto nella variabile p specifica un indizio in memoria fisica oppure in memoria virtuale?

Specifica un indirizzo di memoria virtuale che in fase di esecuzione verranno mappati nei corrispondenti indirizzi di memoria fisica dal Memory Management Unit.

2. Definire cos'è un Sistema di Elaborazione dell'Informazione.

Un Sistema di Elaborazione dell'Informazione è una macchina (digitale, automatica, elettronica, programmabile) in grado di elaborare dei dati.

Quali sono i componenti HW di un computer?

CPU esegue istruzioni, effettua calcoli, controlla input/output, coordina spostamenti di dati.
BUS trasferisce i dati tra la memoria e i dispositivi.
Memoria RAM mantiene programmi quando devono essere eseguiti.
Memoria ROM contiene il BIOS
Dischi mantengono enormi quantità di dati e programmi anche dopo lo spegnimento
Periferiche I/O

3. Che cosa accade durante in bootstrap? E dove si trovano le istruzioni che guidano il bootstrap?

L'avvio del Bootstrap si divide in due fasi:
1)La macchina si accende eseguendo il programma di bootstrap che è già in memoria. Il sistema operativo è memorizzato nella memoria di massa.
2)Il programma di bootstrap dirige il trasferimento del s.o. nella memoria principale e poi le trasferisce il controllo.

4. Una stringa è collocata in memoria in modo diverso se la CPU è di tipo Big Endian invece che Little Endian?

Sì, In Little Endian i Byte più significativi del numero sono collocati dei Byte di indirizzo maggiore.
In Big Endian I Byte più significativi del numero sono collocati nei Byte di indirizzo minore.

5. Dichiarare una struttura dai pacchettizzata.

Una struttura dati di questo tipo è un struttura che evita lo spreco in memoria di spazio occupando solo lo spazio necessario per contenere i singoli membri della struttura e non a multipli di WORD.

```
struct struttura {
     uint32_t i;
     uint8_t c;
} _attribute_((packed))STRUTTURA;
```

6. Sapendo che una CPU lavora "a 64 bit" possiamo dire qualcosa a proposito di qualche BUS del computer?
Una Word è un blocco di byte in memoria avente la stessa di dimensione dell'ampiezza di byte del Bus dei Dati. Questo indica quanti byte possono essere trasferiti, al massimo, con una unica operazione del bus tra CPU e memorie.

7. Cos'è e a cosa serve il DMA?
Il DMA è un meccanismo hardware dedicato al trasferimento dati che consente di ridurre al minimo il coinvolgimento della CPU.
Il trasferimento dei dati, dal disco alla memoria, effettuato dal DMA non coinvolge la CPU tranne che:
- Quando la CPU ordina al DMA lo spostamento del blocco dati.
- Quando il DMA avvisa la CPU dell'avvenuto trasferimento.

8. A cosa serve l'opzione -I del gcc?
L'opzione -I serve per indicare la posizione dei file header.

9. A cosa serve l'opzione -L del gcc?
L'opzione -L serve per indicare la posizione delle librerie (statiche o dinamiche).

A cosa serve l'opzione -Wl,-rpath,RunTimeLibDir del gcc?
L'opzione -Wl,-rpath,RunTimeLibDir serve per indicare la posizione delle librerie dinamiche a run-time.

Cos'è il modo Kernel?
Il modo kernel è una modalità di esecuzione che permette di usare funzionalità pericolose del processore. È spezzato un più parti perché ha diversi livelli di privilegio.

10. Cosa sono i Livelli di Privilegio (Rings) della CPU?
Indicano a quale livello del Sistema Operativo ogni processo può accedere in fase di lettura o scrittura e quali operazioni può effettuare.

11. Un Task può avere più stack a sua disposizione? Perché si o perché no?
Sì, perchè è presente uno stack per le funzioni utente nello spazio utente e più stack per l'esecuzione delle system calls nello spazio kernel.

12. Cos'è una System Call?
Le system call sono servizi messi a disposizione in diverse modalità. Le system call sono disponibili mediante chiamate ad interrupt effettuabili in assembly.

Quali sono i registri specializzati?
Registri Specializzati (nome differente a seconda del processore):

- *Extended Instruction Pointer (EIP) serve a formare l'indirizzo in memoria (detto Program Counter (PC)) della prossima istruzione da eseguire. Contiene l'offset della prossima istruzione da eseguire, rispetto all'inizio del segmento di codice.*
- *Program Status (PS) detto anche Registro di Stato (Status Register).*

- *Stack Segment Register (SS). Punta all'inizio dello stack.*
- *Code Segment Register (CS). Punta all'inizio della sezione text, il codice eseguibile.*
- *Data Segment Register (DS). Punta all'inizio della sezione data, con le var globali.*

Descrivere il supporto ai modi di esecuzione della CPU per i processi.
Quando un processo esegue una chiamata ad interrupt, la CPU entra nella modalità di esecuzione kernel e deve utilizzare uno (o più) stack "di sistema" separato rispetto allo stack utilizzato per le normali chiamate a funzione.
Ciò permette al kernel di separare e proteggere i record di attivazione utilizzati nelle chiamate ad interrupt.
Ciascun processo, perciò, mantiene uno stack per ciascun livello di privilegio della CPU.

Che cosa sono i Task?
I processi, gli interrupt e le eccezioni eseguono ciascuno in un proprio contesto denominato Task. I Task mantengono le informazioni sui segmenti di memoria utilizzati e sul valore corrente dei registri principali.

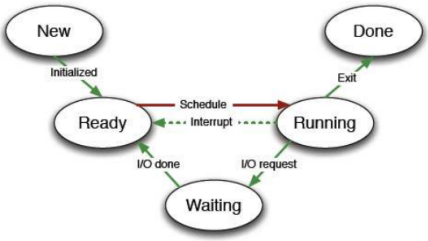
13. Qual'è il compito del loader?
Quando un programma viene lanciato, il loader (che fa parte del sistema operativo) :
- *crea i segmenti di memoria per il processo*
- *carica in memoria il codice eseguibile del programma*
- *carica in memoria la sezione Data*
- *crea lo stack utente per l'esecuzione delle funzioni del programma*
- *crea gli stack di sistema per l'esecuzione delle system calls che verranno invocate dal programma utente*
- *copia eventuali argomenti passati a riga di comando mettendoli a disposizione del main*
- *infine ordina alla CPU di eseguire la prima istruzione eseguibile corrispondente al main del programma*

14. Indicare la differenza tra FAULT e ABORT e dare un esempio per ciascuna delle due eccezioni.
Entrambe sono delle eccezioni.
- *FAULT: l'istruzione che ha causa il trap viene interrotta e viene eseguita la routine di gestione. In un successivo momento l'istruzione verrà rieseguita da principio. (Esempio Page fault, la pagina di memoria che contiene l'indirizzo a cui stiamo cercando di accedere è swappata su disco. La pagina viene caricata in memoria. Prima o poi l'istruzione verrà nuovamente eseguita.)*
- *ABORT: Causata da errore irrimediabile, l'istruzione viene abortita e il processo killato (divisone di un numero per zero, segmentation fault).*

15. Spiegare la differenza tra Interrupt Sincroni e Interrupt Asincroni
Gli interrupt sincroni sono interrupt software esplicitamente chiamati dalla CPU tramite l'istruzione INT n. Quelli asincroni, sono hardware, e sono scatenati da una periferica mediante il BUS che avvisa la CPU che è accaduto un evento da gestire.

Spiegare la differenza tra Interrupt Mascherabili ed Non Mascherabili
- *I Mascherabili possono essere rimandati*
- *Non Mascherabili non possono essere rimandati.*

16. Disegnare il diagramma di stato dei processi, dal punto di vista dello scheduler.



17. Spiegare in che situazione un processo si trova in stato di waiting.
Un processo si trova in stato di waiting il processo effettua una chiamata a sistema bloccante (richiesta I/O).

18. Nella bash, qual'è la differenza tra variabili locali e variabili d'ambiente?
Una variabile locale è una variabile visibile e modificabile solo all'interno della shell che l'ha creata. Mentre una variabile d'ambiente di una shell viene eredita mediante copia dalle sue subshell. Se una subshell modifica tale variabile ereditata comunque non viene modificata solo la copia ereditata.

19. Nella bash, qual'è la differenza tra un variabile vuota e una variabile che non esiste? È possibile eliminare una variabile?
Una variabile vuota è una variabile che è stata creata ma non inizializzata mentre una variabile che non esiste non è ancora stata creata. È possibile eliminarla con il comando unset nomevariabile.

20. Cos'è la funzione di Autocompletamento della bash?
È una funzionalità che mette a disposizione bash per facilitare l'inserimento dei comandi. Per utilizzarla basta scrivere le prime lettere del comando in questione e poi premere TAB.

Descrivere il contesto di un processo.
Ogni processo ha il proprio contesto, ovvero il proprio

- *Process ID,*
- *Program Counter,*
- *Stato dei Registri,*
- *Stack,*
- *Codice,*
- *Dati,*
- *File Descriptors,*
- *Entità IPC,*
- *Azioni dei segnali.*

21. Che cosa hanno in comune i PosiX thread di uno stesso processo?
I PosiX thread di uno stesso processo hanno in comune le variabili globali.

22. Se un processo possiede diversi PosiX thread e quel processo genere un processo figlio, il processo figlio possiede dei thread?
Si, il processo figlio eredita una copia identica dell'ambiente di lavoro del processo padre a partire dal momento della creazione. Il processo figlio eredita un solo thread dal genitore, cioè il thread che aveva chiamato la fork().

23. Se un processo possiede diversi posix thread e quel processo genere un processo figlio, i thread del processo figlio "vedono" le variabile globali del processo padre oppure no?
In generale si può dire che le variabili globali non sono condivise fra processo padre e processo figlio, tuttavia il sistema operativo in realtà condivide inizialmente le pagine, perché fork() implementa un meccanismo copy-on-write, il che significa che a condizione che nessuno dei processi modifichi le pagine, esse sono condivise.

24. Che cos'è errno?
errno è una variabile di errore che ogni thread può avere. Contiene un valore significativo solo se la chiamata è stata effettuata con un errore. Per utilizzare questa variabile dobbiamo far uso della libreria errno.h.

Descrivere i Vantaggi e gli Svantaggi dei Thread.
Vantaggi:
- *Visibilità dei dati globali: condivisione di oggetti semplificata.*
- *Più flussi di esecuzione*
- *gestione semplice di eventi asincroni (I/O per esempio)*
- *Comunicazioni veloci. Tutti i thread di un processo condividono lo stesso spazio di indirizzamento, quindi le comunicazioni tra thread sono più semplici delle comunicazioni tra processi.*
- *Context switch veloce. Nel passaggio da un thread ad un altro di uno stesso processo viene mantenuto buona parte dell'ambiente.*

Svantaggi:
- *Concorrenza invece di parallelismo. Occorre gestire la mutua esclusione per evitare che più thread utilizzino in maniera scoordinata i dati condivisi, modificandoli in momenti sbagliati.*

25. Se un primo thread esegue una chiamata a funzione di libreria del C, e questa produce un errore e scrive il codice di errore nella variabile errno, un altro thread può controllare il contenuto della variabile errno per conoscere il tipo di errore capitato nel primo thread?
No, la variabile errno è specifica per ogni thread

26. Definire la Liveness.
Per Liveness si intende la capacità di un programma di concorrente di far "vivere" e progredire nel tempo i propri thread fino al compimento delle operazioni relative, evitando condizioni che possano determinarne un blocco (ad esempio Deadlock e Starvation).

27. Definire il Busy Waiting.
Il Busy Waiting è un meccanismo di sincronizzazione nel quale un thread che attende il verificarsi di una condizione, lo fa verificando continuamente se la condizione sia diventata vera.

28. Definire il Deadlock.
Stallo o blocco. Due o più thread aspettano indefinitamente un evento che può essere causato solo da uno dei thread bloccati.

29. Definire la Starvation.
Con Starvation si intende l'attesa indefinita da parte di un processo di accedere ad una risorsa perché essa viene continuamente detenuta da processi con priorità maggiore.

30. Che cosa sono i processi Zombie?
Un processo zombie è un processo informatico che, nonostante abbia terminato la propria esecuzione, possiede ancora un PID ed un Process control block, necessario per permettere al proprio processo padre di leggerne il valore di uscita.. Fintanto che il processo padre non esegue la wait, il processo figlio viene definito zombie.

31. Descrivere concisamente qualche tipo di strumento messo a disposizione dallo standard POSIX per sincronizzare tra loro i processi.

Le IPC a memoria condivisa (shared memory) forniscono ai processi cooperanti una porzione di memoria in comune in cui possono scrivere e leggere in modo da passarsi dei dati.
Le IPC a scambio di messaggi (message passing) forniscono ai processi la possibilità di inviare e ricevere messaggi con cui scambiarsi i dati di interesse.

Cos'è il PCB?
Un sistema operativo ha strutture dati apposite con cui tiene traccia di tutti i processi in esecuzione e del loro stato. In particolare per ogni processo viene creato un Process Control Block (PCB) o task control block ove si mantengono informazioni specifiche sul processo e il suo stato.

32. Spiegare cos'è la rilocazione statica.
È l'operazione svolta dal linker che modifica le istruzioni di salto cambiando l'operando che specifica l'indirizzo a cui saltare per specificare correttamente l'indirizzo in cui si trova l'istruzione da eseguire ora che i diversi moduli sono stati accorpati. La rilocazione statica riguarda tutti i salti assoluti.

33. Spiegare che cos'è il binding degli indirizzi.
Si definisce binding degli indirizzi la sequenza con cui ad un indirizzo logico, specificato dalla coppia <SEGMENTO:OFFESET>, viene associato l'indirizzo fisico in memoria RAM fisica. Nei sistemi moderni viene effettuato utilizzando i meccanismi di segmentazione, paginazione e memoria virtuale.

TEORIA SISTEMI OPERATIVI
YYY02

1. In linguaggio ANSI C, siano dichiarate le seguenti variabili, e siano queste variabili debitamente inizializzate:

```
pthread_mutex_t mutex;
pthread_cond_var cond;
```

considerare poi la seguente porzione di codice, in cui condizione sia una espressione da valutare:

```
while (condizione) {
     pthread_cond_wait (&cond, &mutex);
}
fail_qualcosa();
```

La porzione di codice, sopra riportata, effettua busy-waiting?
No, perché wait interrompe l'esecuzione rilasciando la mutua esclusione e mettendosi in attesa del segnale specificato da cond.

2. Una stessa struttura dati ha la stessa dimensione in tutti i processori? Perché?
No, perché dipende da processore e molto spesso i registri general purpose dei processori hanno la stessa dimensione del Bus dei Dati.

3. In un programma in C scrivo una funzione in cui dichiaro un vettore di elementi, i cui elementi sono un tipo di struttura dati struct A. Guardando come viene collocato in memoria quel vettore, può accadere che tra un elemento ed il successivo, dello stesso vettore, ci siano dei byte non occupati dalla struttura dati di tipo struct A? Perché?
Si, se la dimensione della struttura non è un multiplo di una word.

4. In linguaggio C, se un programma è costituito da più moduli, è possibile che una stessa funzione (cioè una funzione con stesso nome, stessi tipi di argomenti e stesso tipo di dato restituito) sia implementata in due diversi moduli senza che questo provochi un errore nella generazione del programma eseguibile? Perché?
Sì, è sufficiente che le funzioni che sono implementate in moduli differenti siano dichiarate "static" così da ridurne lo scope al solo modulo a cui appartengono.

5. La seguente istruzione si trova nella funzione main di un modulo scritto in C.

```
ris = PRODOTTO (3-4, SOMMA(2,7));
```

Precedentemente, nello stesso file, le due macro sono state così definite:

```
#define PRODOTTO (X,Y) ((X)*(Y))
#define SOMMA (X,Y) ((X)+(Y))
```

Se quel modulo viene processato dal preprocessore del compilatore gcc, in che modo viene tradotto quella istruzione dal preprocessore?
ris = ((3-4)((2)+(7)))*

6. Supponiamo di inserire le seguente due istruzioni, in un modulo scritto in C, in una zona fuori da tutte le funzioni del modulo;

```
#define NUM 10
int vet[10];
```

Se quel modulo viene processato dal preprocessore del compilatore gcc, in che modo viene tradotta quella istruzione dal preprocessore?
Il preprocessore sostituisce tipograficamente, ogni volta che compare, 10 al posto di NUM, ma in questo caso non fa nulla perché non compare mai.

7. Spiegare cosa si intende con Race Condition.
Indica la situazione in cui più thread o processi concorrenti accedono (e manipolano) dati condivisi senza controlli di sincronizzazione. In una race condition, il contenuto finale dei dati condivisi dipende dalla sequenza di esecuzione e può causare una inconsistenza dei dati dal punto di vista del singolo thread.

8. Spiegare cosa si intende quando si dice che una sequenza di istruzioni viene eseguita in maniera "atomica".
Una operazione si dice atomica se è indivisibile, ovvero se nessun'altra operazione può cominciare prima che la prima sia finita, è quindi non può esserci interleaving. Il risultato di quella operazione è sempre lo stesso se parte dalle stesse condizioni iniziali.

9. Quando un processo esegue una fork, le sue pagine in memoria vengono immediatamente duplicate?
fork() implementa un meccanismo copy-on-write, il che significa che a condizione che nessuno dei processi modifichi le pagine, esse sono condivise. Un processo o task può avere più thread e ad ognuno di essi è associato uno stack.

10. Spiegare come dovrebbe funzionare l' "algoritmo ottimale" di sostituzione delle pagine.
L'algoritmo ottimale è quello che rendere minimo il numero di page fault che avvengono complessivamente. L'algoritmo funziona scaricando la pagina che non verrà utilizzata per il periodo di tempo più lungo. È però impossibile da implementare perché richiede conoscenza dei riferimenti richiesti nel futuro. Viene utilizzato come lower bound.

11. Spiegare come funziona l'algoritmo "Least Recently Used (LRU)" di sostituzione delle pagine.
L'algoritmo opera in modo da sostituire la pagina che è stata acceduta meno recentemente con la nuova pagina che bisogna caricare in memoria.

12. Consideriamo una finestra del working set di ampiezza 6 e consideriamo un programma che
accede alle proprie pagine in questa sequenza: [3, 5, 7, 2, 8, 5, 6, 5, 4, 5, 6, 7, 1, 5, 6, 4].
Supponiamo ora che sia stata appena acceduto l'ultima pagina della sequenza indicata.
Indicare qual'è il working set.
Il working set è l'insieme dei riferimenti delle pagine accedute negli ultimi 6 accessi.
Risposta: {1, 4, 5, 6, 7}

13. Che cos'è il thrashing?
È uno stato in cui un processo si può trovare quando una allocazione di frame viene gestita con sostituzione globale. Il processo sta perdendo più tempo in attività correlate alla paginazione che in attività di calcolo.

14. Nell'ambito della paginazione per la memoria virtuale, cosa si intende con "allocazione dei frame ai processi"?
Per allocazione dei frame ai processi si indica il processo per riservare lo spazio in memoria che deve essere assegnata ai processi per rimanere in esecuzione.

15. Spiegare a cosa serve il procedimento di sostituzione delle pagine nell'ambito della gestione della memoria virtuale.
In un sistema con memoria virtuale basata su paginazione su richiesta è importante tenere basso il numero di page fault altrimenti il tempo effettivo di accesso aumenta a dismisura e si rallenta notevolmente l'esecuzione dei processi. Per questo è cruciale scegliere bene il meccanismo di sostituzione delle pagine. Se un processo utilizza solo una parte delle pagine, la paginazione su richiesta consente di eliminare i tempi di caricamento delle pagine che non verranno utilizzate.

16. In che contesto si parla di "dirty bit" e a che cosa serve?
È un bit riservato all'interno della pagina per indicare se quest'ultima è stata modificata o Meno.
Nell'ambito del procedimento di sostituzione delle pagine quando viene selezionata una pagina per la sostituzione, se la pagina è stata modificata quella pagina deve essere scritta sul disco.
Invece se la pagina non è stata modificata e se la copia della pagina sul disco non è stata sovrascritta (da qualche altra pagina, ad esempio), allora e possibile evitare di scrivere la pagina di memoria sul disco.

17. Spiegare cosa avviene allo scatenarsi di una page fault.
Quando avviene un page fault è perché il processo in questione non è riuscito a trovare la pagina richiesta in memoria fisica, quindi viene sospeso e avviene una sostituzione delle pagine, ossia viene caricata in memoria fisica da disco la pagina necessaria.

18. Spiegare cosa sono i processi "I/O bound" e quelle "CPU bound".
In un processo:
- *I/O bound I cicli di CPU burst sono molti ma molto brevi perché frequentemente interrotti da attività di I/O*
- *CPU bound I cicli di CPU burst sono meno ma più duraturi.*

19. Spiegare la differenza tra "scheduling preemptive" e "scheduling non preemptive".
È importante la distinzione tra scheduling con diritto di prelazione (preemptive) e quello senza (non preemptive). Nel primo caso se il processo rientrato ha priorità maggiore di quello in esecuzione prima del rientro, lo spodesta. Lo scheduler entra in funzione ogni rientro nella ready queue. Nel secondo caso lo scheduler deve attendere che il processo termini o che cambi il suo stato da quello di esecuzione a quello di attesa o di pronto, a seguito, ad esempio, di una richiesta di I/O o a causa di un interrupt.

20. L'algoritmo di scheduling FIFO soffre di starvation? Perché?
No l'algoritmo di scheduling FIFO non soffre di starvation può soffrire al più di effetto convoglio.

21. Cos'è la formattazione fisica o formattazione a basso livello?
L'operazione con cui il disco viene suddiviso in settori

22. Spiegare come avviene il bootstrap nei sistemi moderni.
Il bootstrap viene effettuato in due fasi.
Nella prima viene letto in piccolissimo programma all'interno della ROM, detto bootstrap loader, che avvia il boot che è il programma completo di avvio del Sistema Operativo che si trova nell'area di boot del disco.

23. A cosa serve il programma fdisk?
fdisk è un programma basato su menu per la creazione e la manipolazione di tabelle di partizione.

24. A cosa serve il programma ln?
È un programma che consente di creare diversi tipi di collegamenti a file esistenti (Hard Link e Soft Link).

25. Devo implementare una delle politiche di scheduling del disco magnetico descritte a lezione, ma ancora non so quale. Che informazioni mi servono per poter implementare una qualunque di quelle politiche?
Il metodo scelto per l'allocazione dei file, La posizione in cui sono memorizzate le directory.

26. Perché non parliamo di politiche di scheduling del disco quando abbiamo a che fare con dischi a stato solido?
Perché a differenza dei dischi meccanici non ho parti in movimento quindi le prestazioni del disco sono in realtà caratterizzate solo dalla velocità di trasferimento e non anche da quella di posizionamento della testina.

27. Ipotizziamo che i cilindri di un disco magnetico siano indicizzati in maniera crescente dalla periferia verso il centro.
Supponiamo che la testina di lettura di un disco si trovi sul cilindro di indice 10. Ipotizziamo che la testina, per arrivare alla posizione attuale, si sia spostata verso il centro del disco.
Supponiamo che l'elenco delle richieste pendenti sia la seguente, ordinata in ordine di tempo di arrivo delle richieste.
12, 15, 6, 3, 6, 11, 13, 17, 20, 1.
Se uso un algoritmo di scheduling Shortest Seek Time First (SSTF), quale sara' la sequenza temporale dei prossimi accessi a disco?
Risposta: 11, 12, 13, 15, 17, 20, 6, 6, 3, 1

28. Nelle stesse ipotesi dell'esercizio precedente, quale sarà la sequenza temporale dei prossimi accessi a disco se invece uso una politica di scheduling del disco di tipo C-LOOK?
Risposta: 11, 12, 13, 15, 17, 20, 1, 3, 6, 6

29. Spiegare la differenza tra Hard link e Soft link in ambito Linux.
Gli Hard Link hanno nomi diversi ma puntano allo stesso inode, i Soft link sono un tipo speciale di file che si riferiscono ad un file (inode) diverso.

30. Cos'è un inode?
È una struttura dati sul file system che archivia e descrive attributi base su file, directory e contiene Nome, Proprietario, Dimensione, Permessi di accesso, posizione dei blocchi.

31. Che struttura si utiliza, in ambiente Linux, per memorizzare i blocchi di un file?
L'inode è una struttura che contiene gli attributi e le caratteristiche del file a cui è legato (i-node sta per information node, cioè nodo di informazione). Esiste un i-node per ogni file.

~~TEORIA SISTEMI OPERATIVI~~

1. **Cosa sono le API?**
   Le API (Application Program Interface) sono interfacce che vengono offerte dal SO ai programmi di alto livello per utilizzare le system calls. Le system call cambiano da SO a SO e quindi cambia il modo di implementazione delle funzioni API per adattarle al sistema operativo ma i nomi delle funzioni rimangono sempre gli stessi.

2. **A cosa serve il comanda *strace* da linea di comando?**
   Il comando strace serve a vedere quali system call sono usate da un programma in esecuzione.

3. **Descrivi gli step del modello di compilazione per il C.**
   Preprocessor: processa le direttive al preprocessore (tutti i comandi preceduti dal pound #) ed elimina tutti i commenti.
   Compiler: prende in input l'out del preprocessore e lo trasforma in un codice assembly.
   Assembler: trasforma il codice assembly in un modulo oggetto.
   ~~*Linker: prende in input i file oggetto e li collega per generare il file eseguibile.*~~

4. **Definisci i vari tipi di variabile.**
   Locali: ossia la sua durata di vita è pari alla durata di vita del blocco di istruzioni in cui è dichiarata
   Formali: sono le variabili che vengono passate alle funzione.
   Globali: sono quelle variabili che sono dichiarate fuori da tutte le funzioni. Una variabile di questo tipo può essere acceduta da tutte le funzioni di quel file e da tutte quelle in altri file in cui esiste una dichiarazione extern per la stessa variabile. Per proteggere una variabile da accessi ~~*esterni necessita della dichiarazione static.*~~

5. **Cos'è struttura dai pacchettizzata.**
   Una struttura dati di questo tipo è un struttura che evita lo spreco in memoria di spazio occupando solo lo spazio necessario per contenere i singoli membri della struttura e non a ~~*multipli di WORD.*~~

6. **Una stessa struttura dati ha la stessa dimensione in tutti i processori? Perché?**
   No, perché dipende quanto sono grandi i tipi di dato di quel processore (un int non è detto che ~~*abbia la stessa per tutti i processori 64/32 bit), da se pacchettizza o meno le strutture dati…*~~

7. **In un programma in C scrivo una funzione in cui dichiaro un vettore di elementi, i cui elementi sono un tipo di struttura dati struct A. Guardando come viene collocato in memoria quel vettore, può accadere che tra un elemento ed il successivo, dello stesso vettore, ci siano dei byte non occupati dalla struttura dati di tipo struct A? Perché?**
   ~~*Si, se la struttura dati non è pacchettizzata, cioè la sua dimensione è un multiplo di WORD.*~~

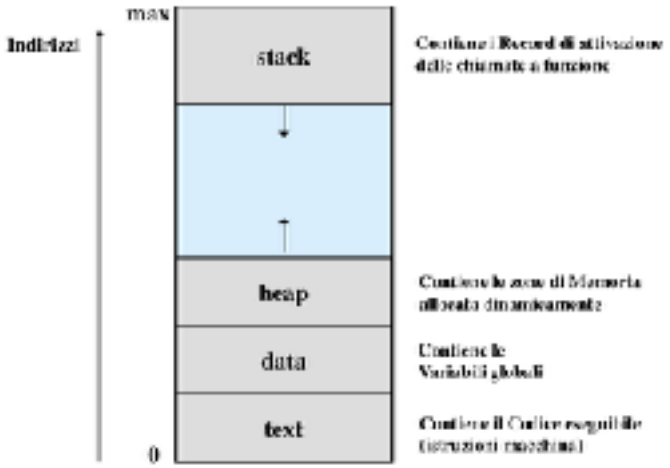
8. **In linguaggio C, se un programma è costituito da più moduli, è possibile che una stessa funzione (cioè una funzione con stesso nome, stessi tipi di argomenti e stesso tipo di dato restituito) sia implementata in due diversi moduli senza che questo provochi un errore nella generazione del programma eseguibile? Perché?**
   No, non è possibile perché in fase di linking viene segnalata la ridefinizione. Lascia però compilare perché preso il modulo singolarmente è valido. Però se in uno dei due moduli tale funzione è dichiarata static non è una ridefinizione in quanto static fa in modo che sia una cosa interna solo al quel modulo cioè invisibile ad altri.

9. A cosa serve l'opzione -I del preprocessore e -L e -l del compilatore?
-I specifica il percorso dei file .h.
-L specifica il percorso delle librerie fornite dall'utente.
-l nomelibreria *specifica di utilizzare la libreria indicata dal nome ristretto* nomelibreria

10. Differenza tra librerie statiche e dinamiche.
Librerie statiche: collegate al programma PRIMA della sua effettiva esecuzione, nella fase di preprocessing dove prende il file.h e lo copia dove c'è il l'include.
Librerie dinamiche: vengono collegate in due fasi. La prima controlla che si possa effettivamente collegare le librerie richieste al programma (compiler time). La seconda a run time, un programma dedicato le carica in memoria e le collega al programma in esecuzione.

11. Che cos'è un target fittizio?
Un target fittizio, altrimenti detto .phony, è un target che non è un file e ha come scopo solo l'esecuzione di una sequenza di azioni. Bisogna indicarli tutti in fondo al Makefile ed elencanti prima come .PHONY: nomi-target.

12. Organizzazione della memoria usata dalla parte utente dei processi.



—> *tutto ciò che serve ad una funzione*

13. Che cos'è un *Task*?
Un Task, è il contesto in cui eseguono i processi. Ogni processo ha un proprio Task. Un Task mantiene in memoria lo stato di esecuzione di un processo, le informazioni sui segmenti di memoria utilizzati e sul valore corrente dei registri principali.

14. Un Task può avere più stack a sua disposizione? Perché si o perché no?
SI, perché ogni livello di privilegio del SO gode di un suo stack di esecuzione, in questo modo ogni livello viene mantenuto come componente atomico che non influenza gli altri. Perché ogni Task viene differenziato nei vari livelli a cui opera. (così operazioni indipendenti non interferiscono tra di loro)

15. Che differenza c'è tra un programma e un processo?
Un processo è l'istanziazione in memoria di un programma (file binario eseguibile). Un programma può essere:
- un file binario eseguibile
- un processo
- una applicazione (cioè un insieme di processi)
Inoltre un processo è un'entità attiva mentre un programma è un'entità passiva

16. Quanti stack ha a disposizione un processo?
Un processo ha a disposizione uno stack per ogni livello di privilegio del kernel (4) più uno per ogni thread del processo.

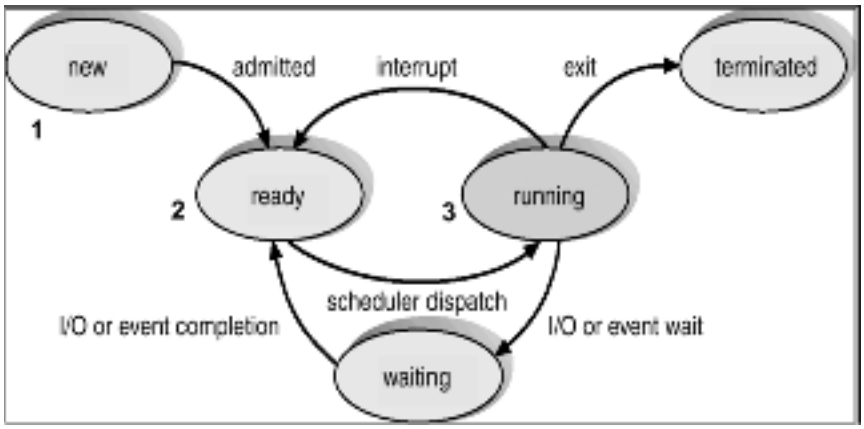
17. Cos'è una System Call?
Chiamata a Sistema, avviene quando un processo fa una richiesta al kernel, in quanto una determinata operazione non sarebbe possibile; al termine dell'operazione a livello kernel il controllo viene riconsegnato al processo che ne ha fatto richiesta.

18. Descrivi il meccanismo delle System Calls.
Una system call è implementata nella routine di gestione di un certo interrupt n. Viene invocata eseguendo una istruzione INT n.

19. Interrupt e Eccezioni.
Le ECCEZIONI sono sincrone rispetto alla CPU (ossia chiamate direttamente dalla CPU)
- TRAP: l'istruzione viene sospesa e portata a buon fine solo dopo la gestione del trap (trap per debugging)
- FAULT: l'istruzione viene interrotta, gestita l'eccezione e poi rieseguita l'istruzione dall'inizio (Page Fault, ossia l'impossibilità da parte di un processo di accedere ad una pagina presente nel suo spazio di indirizzo virtuale ma che non è presente nella memoria fisica. Il Sistema cerca di caricare tale pagina e poi fa ripartire il processo)
- ABORT: viene causata da un errore irrimediabile, quindi l'istruzione viene abortita e il processo killato (segmentation fault: avviene quando un processo tenta di accedere ad un'area di memoria a cui non gli è permesso accedere quindi potrebbe danneggiare altri dati)
Gli INTERRUPT possono essere sincroni e asincroni (dovuti dall'hardware), mascherabili e non mascherabili (ossia possono essere ignorati o meno).

20. Spiegare la differenza tra Interrupt Sincroni e Interrupt Asincroni
Gli Interrupt Sincroni (sw) sono esplicitamente chiamati dalla CPU, mentre gli Interrupt Asincroni (hw) sono scatenati dalle periferiche che mediante il BUS avvisa la CPU che è avvenuto qualcosa da gestire (la tastiera avvisa che l'utente ha premuto un tasto).

21. Disegnare il diagramma di stato dei processi, dal punto di vista dello scheduler.
Lo scheduler è quel programma che processa le richiesti di accesso ad una risorsa condivisa, cioè stabilisce un ordine per l'esecuzione di tali richieste.
Lo SHEDULER A LUNGO TERMINE seleziona i processi che possono entrare nella ready queue. Mentre lo SCHEDULER A BREVE TERMINE seleziona quali processi possono passare dallo stato di ready allo stato di running.
New —(loader lo carica)—> Ready(pronto per l'esecuzione) —(esecuzione)—(exit)—>termina
N.B. durante la fase di running può passare dalla fase di Waiting a causo di tempo scaduto o attesa di input/output.



22. **Nella bash, qual'è la differenza tra variabili locali e variabili d'ambiente?**
Una variabile locale è una variabile visibile e modificabile solo all'interno della shell che l'ha creata. Mentre una variabile d'ambiente di una shell viene eredita mediante copia dalle sue subshell. Se una subshell modifica tale variabile ereditata comunque non viene modificata quella d'ambiente ma solo la copia ereditata.

23. **Nella bash, qual'è la differenza tra un variabile vuota e una variabile che non esiste? È possibile eliminare una variabile?**
Una variabile vuota è una variabile che è stata creata ma non inizializzata mentre una variabile che non esiste non è ancora stata creata. È possibile eliminarla con il comando unset nomevariabile.

24. **Cos'è la funzione di Autocompletamento della bash?**
È una funzionalità che mette a disposizione bash per facilitare l'inserimento dei comandi. Per utilizzarla basta scrivere le prime lettere del comando in questione e poi premere TAB.

25. **Che cos'è un Thread?**
È un flusso di esecuzione all'interno di un processo che lo scheduler può fare eseguire separatamente e concorrentemente con il resto del processo. Ogni thread di uno stesso processo condivide le risorse del processo. Ad esempio vede tutte le variabili globali del processo che l'ha creato.

26. **Che cos'è errno?**
Errno è una variabile numerica che contiene il tipo di errore che è stato provocato per ultimo/ dall'ultima istruzione.

27. **Spiegare cosa si intende quando si dece che una sequenza di istruzioni viene eseguita in maniera "atomica".**
Vuole dire che una sequenza di istruzioni o viene eseguita per intero oppure non viene proprio eseguita. Se avviene un errore durante l'esecuzione (in mezzo) si riparte da capo.
Il compilatore gcc fornisce alcune estensioni di C (Atomic Built-in function) che permettono di eseguire in forma atomica alcune operazioni. Ad esempio per la mutua esclusione _sync_bool_compare_and_swap. *Tale funzione viene utilizzata per verificare che il NuovoValore sia stato scritto o meno. Ritorna true in caso di successo false altrimenti.*
N.B. È implementata mediante l'istruzione ISA lock comxchg.

28. **Cosa vuol dire che una funzione è thread safe?**
Una funzione è thread safe se non causa problemi nella scrittura/lettura di strutture dati interne al so e condivise dai thread di uno stesso processo. Alcune funzioni di libreria non sono rientranti, per usare la forma rientrante bisogna usare la funzione nomefunzione_r.

29. **Che cosa si intende con Pthread?**
Un Pthread è un thread conforme allo standard POSIX, che si avvalgono di funzione di mutex (mutua esclusione) e condition variables.
Per la creazione di un Pthread si utilizza la funzione pthread_create

30. **A cosa serve la funzione pthread_join?**
Serve ad liberare la memoria di un thread una volta che esso è terminato facendo sapere il proprio risultato.

31. **Che cos'è un Pthread *detached*?**
È un Pthread che è in grado di liberare autonomamente la memoria una volta terminato ma in questo modo non può far sapere a nessuno il proprio risultato.

32. Definire il Busy Waiting.
Con Busy Waiting ci si riferisce al continuo controllo da parte di un thread che un evento si sia verificato per permettergli di proseguire restando in stato di ready e non in wait. È da evitare in quanto comporta uno spreco di tempo e prestazioni della CPU.

33. Definire il Deadlock.
Una situazione di Deadlock avviene quando due o più processi si bloccano a vicenda.
Un esempio è rappresentato da due persone che vogliono disegnare. Per disegnare hanno a disposizione solo una riga e una matita. Per disegnare hanno bisogno di entrambe. Potendo prendere un solo oggetto per volta, se uno prende la matita e l'altro prende la riga, e se entrambi aspettano che l'altro gli dia l'oggetto che ha in mano, i due generano un Deadlock.

34. Definire la Starvation.
Con Starvation si intende l'impossibilità perpetua da parte di un processo di accedere ad una risorsa di cui necessita per essere eseguito.

35. Definire la Liveness.
Per Liveness si intende la capacità di un programma di concorrente di far "vivere" e progredire nel tempo i propri thread fino al compimento delle operazioni relative, evitando condizioni che possano determinarne un blocco (ad esempio Deadlock e Starvation).

36. Spiegare cosa si intende con Race Condition.
Per Race Condition è una situazione che si verifica in un sistema con processi e/o thread quando questo non gestisce in modo adeguato la mutua esclusione. Pertanto il risultato finale dipende esclusivamente dall'ordine in cui sono state eseguite le istruzioni.

37. Definire cosa fa la funzione cmpxchg con e senza il prefisso lock.
Tale funzione è l'equivalente di sync_bool_compare_and_swap in linguaggio assembly (in effetti è utilizzata nell'implementazione della funzione C sync_bool_compare_and_swap). Ossia se la variabile è diversa dal valore che gli si vuole assegnare le si assegna il nuovo valore altrimenti nulla. Il prefisso lock garantisce che per tutta la durata dell'esecuzione dell'istruzione nessuno acceda al bus per far si che nessuno vada a modificare i registri che servono per la comparazione.

38. Che cos'è la *Futex*?
La Futex (Fast User Mutex) è una forma base di mutua esclusione e non è conforme a nessuno standard POSIX.

39. Quali sono i possibili formati di un file eseguibile in ambiente Unix/Linux e Windows?
In ambiente Unix/Linux sono a.out (vecchio) e ELF (di default sui sistemi Linux). Su Windows è PE (Portable Executable).

40. Che cos'è il PCB (Process Control Block)?
È una tabella dove vengono mantenute le informazioni sul processo e sul suo stato.
Ogni PCB si riferisce ad un unico processo ed è memorizzato in una struttura dati del kernel detta TABELLA DEI PROCESSI.

41. Che cos'è la Tabella dei Processi?
È una struttura dati con cui il kernel tiene traccia di tutti i processi attivi nel sistema. Ogni entry della tabella è il PCB di un processo attivo. Il kernel mantiene anche aggiornato un puntatore current*, che punta al descritto del processo correntemente in esecuzione.*

42. Differenza tra *multi-threading* e *multi-processing*?
La differenza è che i thread di uno stesso processo condividono la stessa area di memoria. Mentre i processi figli hanno ognuno un proprio spazio di indirizzamento di memori (ossia non c'è condivisone di memoria tra i processi).

43. Come vengono creati i processi figli?
Un processo figlio viene creato dal processo padre mediante la funzione fork(). Il processo figlio ha uno spazio di memoria che è una copia esatta di quello del padre.

44. Quando un processo esegue una fork, le sue pagine in memoria vengono immediatamente duplicate?
No, perché non ha senso fare n copie della stesso processo padre, viene semplicemente mantenuto un riferimento alle pagine del processo padre. Le sue pagine verranno duplicate nel momento in cui verrà effettivamente fatta una modifica da uno dei due processi.

45. Che cosa sono i processi Zombie?
I processi Zombie sono quei processi che pur essendo terminati continuano ad occupare memoria inutilmente. È per questo che il processo padre una volta che i processi figli terminano deve pensare a deallocare totalmente la memoria da loro occupata, ossia ad ucciderli, facendo una wait. Se il processo padre termina senza eseguire la wait allora i suoi processi figli si dicono Orfani. In tal caso il processo orfano viene adottato dal processo init *(quello che ha originato tutti i processi al boot) che ogni tanto effettua delle wait.*

46. Che cosa si intende con IPC (Inter-Process Comunication)?
Sono meccanismi di comunicazione di cui necessitano i processi per scambiarsi dati e sincronizzazioni. Esistono due tipi di IPC:
- a memoria condivisa: ossia forniscono ai cooperanti una porzione di memoria in comune in cui possono scrivere e leggere in modo da passassi dati.
- a scambio di messaggi: forniscono ai processi la possibilità di inviare e ricevere messaggi con cui scambiarsi i dati di interesse (a distanza).

47. Che cos'è il Binding Address o rilocazione?
È l'associazione di indirizzi di memoria fisica ai dati e alle istruzione di un programma (indirizzi virtuali).
Può avvenire:
- durante la compilazione, allora gli indirizzi rimarranno invariati ad ogni esecuzione e parliamo di codice assoluto
- durante il caricamento, cioè è il loader a aggiornare tutti i riferimenti coerentemente alla punto iniziale di caricamento (codice rilocabile)
- durante l'esecuzione, allora l'indirizzo di memoria effettivo viene calcolato durante l'esecuzione dalla MMU che prende in input un indirizzo logico e restituisce l'indirizzo fisico corrispondente.

48. Che cosa si intende per "allocazione della memoria" e che tipi di allocazione esistono? Descrivili.
L'allocazione della memoria consiste nell'assegnare una spazio di memora fisica ad un programma. L'allocazione può essere:
- CONTIGUA: se tutto lo spazio assegnato ad un programma deve essere formato da celle consecutive.
- NON CONTIGUA: se è possibile assegnare aree di memoria separate.
- STATICA: ossia il programma deve mantenere l'area di memoria che gli è stata assegnata per tutta la durata della sua esecuzione.

- DINAMICA: quando durante l'esecuzione un programma può essere spostato all'interno della memoria.
- A PARTIZIONI FISSE: ossia, la memoria viene divisa in partizione a dimensione prefissata e un processo viene allocato in una partizione libera sufficientemente grande. Questo può causare frammentazione interna (quando un processo occupa una dimensione inferiore a quella della partizione).
- A PARTIZIONI DINAMICHE: la memoria non viene suddivisa ma viene assegnata ai processi che ne fanno richiesta. Questo però causa frammentazione esterna (ossia dopo un certo numero di allocazione e deallocazioni lo spazio libero appare suddiviso in piccole aree). Per questo esiste la COMPATTAZIONE, che significa spostare tutti i programmi in modo da riunire tutte le aree inutilizzate.

49. Cos'è e a cosa serve la "mappa di bit"?
Serve ad identificare le unità di allocazione della memoria, ad ogni unità corrisponde un bit nella bitmap. 0 se l'unità è libera 1 se occupata. Un altro modo per identificare quali unità sono libere o meno è con la LISTA DI PUNTATORI, dove ogni elemento della lista specifica se si tratta di un processo (P) o di un blocco libero (H) e l'inizio e la fine del segmento.

50. Definisci i vari metodi di selezione dei blocchi liberi.
First Fit scorre la lista dei blocchi liberi dall'inizio fino a quando non trova il primo segmento vuoto grande abbastanza da contenere il processo.
Next Fit è come First Fit ma al posto che ripartire dall'inizio parte dal punto dove si era fermato all'ultima allocazione.
Best Fit, seleziona il più piccolo fra i blocchi liberi.
Worst Fit, seleziona il più grande fra i blocchi liberi.

51. Nell'ambito della paginazione per la memoria virtuale, cosa si intende con "allocazione dei frame ai processi"?
Per "allocazione dei frame" ai processi si intende il processo di transizione da pagine a freme, ossia il passaggio delle pagine dalla memoria virtuale ai corrispondenti frame della memoria fisica.

52. Descrivi il metodo della allocazione dinamica attraverso l'uso della paginazione.
Con la paginazione la memoria virtuale (ossia lo spazio di indirizzamento logico) viene divisa in blocchi di dimensione prefissata chiamati pagine*, mentre la memoria fisica viene suddivisa in blocchi della stessa dimensione delle pagine chiamati* frame.
Quando un processo deve essere eseguito vengono reperiti un numero sufficiente di frame per contenere le pagine del processo.

53. Come si sceglie la dimensione delle pagine?
La dimensione delle pagine deve essere una potenza di 2, per facilitare il passaggio da indirizzi virtuali (logici) a indirizzi fisici. In genere è di 4KB.

54. Come funziona l'MMU nella paginazione?
L'MMU prende in input un indirizzo logico formato da una coppia di valori del tipo (pagina, offset) e restituisce in indirizzo fisico formato dalla coppia (frame, offset), avvalendosi della tabella delle pagine che si trova in memoria.

55. Che cos'è un Translation Lookside Buffer (TLB)?
Un TLB è un insieme di registri formati da chiave (l'indice della pagina) e valore (l'elemento della tabella delle pagine ossia il frame corrispondente). Questi registri vengono utilizzati per velocizzare il processo di trasformazione di un indirizzo logico ad un indirizzo fisico.
Se la chiave è presente nel registro allora si restituisce il valore corrispondente altrimenti lo si va

a cercare nella tabella della pagine in memoria.
N.B. Accedere alla memoria costa molto di più, la TLB agisce come memoria cache per la tabella delle pagine.

56. Come viene divisa la memoria in un sistema con segmentazione?
La memoria viene suddivisa in aree differenti dal punto di vista funzionale. Un segmento è un'area di memoria contenente informazioni omogenee e il suo riferimento in memoria è dato da una coppia (nome_segmento, offset). A differenza della paginazione la divisione in segmenti del programma spetta al programmatore.

57. Come funziona l'MMU nella segmentazione?
L'MMU prende in input un indirizzo logico formato da una coppia (segmento, offset), ricerca nella tabella dei segmenti l'indirizzo di inizio del segmento e somma l'offset restituendo l'indirizzo fisico corrispondente.

58. Spiega brevemente la segmentazione paginata.
Ogni segmento viene suddiviso in pagine che vengono allocate in frame liberi della memoria non necessariamente contigui.
L'indirizzo logico viene trasformato in un indirizzo virtuale e successivamente nel reale indirizzo fisico.

59. Spiegare a cosa serve il procedimento di sostituzione delle pagine nell'ambito della gestione della memoria virtuale.
Il procedimento di sostituzione delle pagine serve ad avere a disposizione sempre le pagine in memoria virtuale che servono ai processi in esecuzione, per questo si utilizzano vari algoritmi di avvicendamento.

60. A cosa serve la memoria virtuale?
È la tecnica che permette l'esecuzione di processi che non sono completamente in memoria. Permette di eseguire in concorrenza processi che nel loro complesse hanno necessità di memoria maggiore di quella disponibile.

61. Su cosa possono essere mappati gli indirizzi virtuali?
Gli indirizzi virtuali possono essere mappati sia si indirizzi fisici della memoria principale sia su memoria secondaria. In quest'ultimo caso in dati associati in memoria secondaria vengono trasferiti in memoria principale, se non c'è più spazio si sposta in memoria secondaria i dati contenuti in memoria principale che sono considerati meno utili.

62. Che cos'è la "demand paging"?
La demand paging e la paginazione su richiesta che avviene quando un processo tenta di accedere ad una pagina in memoria principale ma non è presente e questo genera un page fault che viene gestito dal pager che si occupa di mettere in memoria principale (swap-in) la pagina mancate e aggiornare la tabella delle pagine cambiando il bit di validità.

63. Che cos'è la *swap area*?
La swap area è l'area del disco utilizzata per ospitare le pagine in memoria secondaria non utilizzate al momento dai processi.

64. Che cosa sono gli algoritmi di rimpiazzamento o avvicendamento?
Sono algoritmi che vengono utilizzati per gestire la sostituzione delle pagine in memoria. Ovvero per scegliere la pagina "vittima" da rimpiazzare (swap-out).

65. Che cosa avviene con la virtualizzazione della memoria?
Con la virtualizzazione della memoria non tutto il programma è nella memoria fisica (RAM) della macchina. Per tale procedure viene virtualizzato una spazio della memoria fisica (principale) maggiore di quello fisicamente disponibile. Per farlo si una uno spazio della memoria secondaria su cui vengono mantenute parte dei processi che non sono indispensabili immediatamente al processo.

66. Che cos'è la mappatura della memoria?
La mappatura della memoria consente al sistema di allocare la memoria condivisa una sola volta e ai processi di vedere librerie caricate dinamicamente e memoria condivisa come parte del proprio spazio degli indirizzi inserendo nello "spazio vuoto" tra stack e heap dei puntatori a tale area di memoria condivisa.

67. Che cos'è il bit di validità?
Il bit di validità è un bit presente nella tabella delle pagine che assume 1 se il frame corrispondente è presente in memoria fisica e allora l'indirizzo logico viene trasformato in fisico, se il bit è 0 allora si genera un page fault e quindi si procede a caricare in memoria la pagina mancante.

68. Che cosa si intende con paginazione su richiesta pura?
È uno schema di esecuzione per avviene quando si avvia un processo senza alcuna pagina in memoria. Quindi alla prima istruzione si genererà immediatamente un page fault.

69. Che cos'è il Backing store?
È una parte del disco (memoria secondaria) che viene utilizzata per la memorizzazione stabile delle pagine. È un sinonimo di swap area.

70. Quali sono gli effetti positivi della paginazione su richiesta?
Se un processo utilizza solo una parte delle pagine, tale procedure consente di eliminare i tempi di caricamento di pagine che non verranno mai utilizzate.

71. Che cos'è la "copy on write"?
Quando si genere un processo figlio mediante la chiamata alla funzione fork(), le pagine del padre andrebbero teoricamente copiate nell'ambiente di esecuzione del figlio. Tale procedimento potrebbe risultare inutile in quanto se il figlia cambia immediatamente il suo ambiente di esecuzione tutte le pagine copiate diventerebbero inutili. Per evitare questo spreco di tempo, si utilizza la tecnica della copiatura su scrittura. Ossia le pagine del padre vengono inizialmente condivise e vengono effettivamente duplicate solo quando il padre o il figlio necessitano di modificare qualcosa.

72. In che contesto si parla di "dirty bit" e a che cosa serve?
È un bit riservato all'interno della pagina per indicare se quest'ultima è stata modificata o meno. Gli algoritmi di avvicendamento prediligeranno una pagina che non è stata modificata da scartare piuttosto che una modificata in quanto il tempo di sostituzione si dimezza non dovendo modificare l'immagine su disco della pagina modificata.

73. Elenca e spiega i vari tipi di algoritmi di sostituzione delle pagine.
<u>FIFO</u>: nel momento in cui si necessità di uno swap-out, si sceglie la pagina con l'istante di caricamento più basso (la più vecchia)
<u>LRU (Least Recently Used)</u>: scarica la pagina che non viene acceduta da più tempo, utilizzando un bit di riferimento (0 se la pagina non è stata acceduta, 1 altrimenti) oppure

un contatore degli accessi.
<u>Bit supplementari di riferimento</u>: ogni N millisecondi viene salvato il bit di riferimento di tutte le pagine (si usa un byte per ogni pagina) e si rialzeranno tutti i bit. Nel momento dello swap-out la pagina usata meno recentemente è quella con il byte di valore più basso.
<u>Seconda Chance</u>: utilizza un solo bit per pagina. Le pagine sono tenute in una coda circolare con un puntatore: se trova un bit a 0 sostituisce se lo trova a 1 lo azzera. In questo modo la pagina con bit 1 ha una seconda chance.
<u>Seconda Chance migliorato</u>: usa la coppia (bit di riferimento, dirty bit). Privilegia lo scaricamento di pagine non modificate per rendere più veloce lo swap-out.

74. Che cos'è l'anomalia di Belady?
È ciò di cui può soffrire un algoritmo nel caso in cui aumentando il numero di frame disponibili al posto che diminuire, i page fault aumentano.

75. Spiegare come dovrebbe funzionare l' "algoritmo ottimale" di sostituzione delle pagine.
L'algoritmo ottimale è l'algoritmo che risolve il problema nel miglior modo possibile. Il problema della sostituzione delle pagine mira a minimizzare in numero dei page fault.
L'algoritmo ottimale è quello che conoscendo la sequenza delle pagine richieste le fa trovare sempre al momento in giusto in memoria centrale (predice il futurooooo).

76. Spiega la differenza tra sostituzione globale e locale.
Si parla di sostituzione globale quando la richiesta di sostituzione è soddisfatta prelevando un frame da un processo qualunque. Locale invece quando viene prelevata dal processo che ha fatto la richiesta.
Nel caso di una sostituzione globale può accadere che un processo con priorità alta porti un processo con priorità inferiore ad un numero di frame inferiore al minimo. Occorre in questo caso richiamare lo scheduling e rimuovere il processo dalla ready queue prima che vada in trashing.

77. Che cos'è il thrashing?
Il thrashing si verifica quando un algoritmo di sostituzione delle pagine decide di scartare una pagina che verrà usata a breve. Quindi vuol dire che il processo si trova a dedicare più tempo alla sostituzione delle pagine che alla effettiva esecuzione di istruzioni. Questo può causare un circolo vizioso in quanto un processo in thrashing non usa CPU, il SO rileva un basso uso della CPU e quindi carica altri processi, questo porta altri processi ad andare in thrashing.

78. Consideriamo una finestra del working set di ampiezza 6 e consideriamo un programma che accede alle proprie pagine in questa sequenza: [3, 5, 7, 2, 8, 5, 6, 5, 4, 5, 6, 7, 1, 5, 6, 4]. Supponiamo ora che sia stata appena acceduto l'ultima pagina della sequenza indicata. Indicare qual'è il working set.
Il working set è l'insieme dei riferimenti delle pagine accedute negli ultimi 6 accessi.
Risposta: {7, 1, 6, 5, 4}

79. Che cos'è il working set?
È l'insieme delle pagine che contengono gli ultimi x riferimenti.

80. Che cos'è e a cosa serve un file mappato in memoria?
Un file mappato è un file creato u disco con un riferimento ad un area di memoria. I processi accedono in modo indiretto a tale area grazie al file mappato su disco a cui possono accedere tutti. È usato come salvataggio dell'area di memoria.

81. Che cosa sono i registri STLR (Segment Table Limit Register) e STBR (Segment Table Base Register)?
STBR contiene l'indirizzo di inizio della tabella dei segmenti del processo in esecuzione.
STLR contiene la dimensione della tabella.

82. Quali indirizzi vengono specificati dalle istruzioni di un programma in esecuzione?
Virtuali.

83. Quali indirizzi vengono spediti sul bus degli indirizzi?
Fisici

84. Se debuggo un programma che usa puntatori, che indirizzi vedo nelle variabili puntatori?
Virtuali.

85. A cosa serve il comando sysctl -w kernel.randomize_va_space = 0?
Serve a far si che il kernel non randomizzi l'indirizzamento fisico perciò ad esempio nel lanciare uno stesso programma più volte che stampa un indirizzo avrò sempre lo stesso output.

86. Che cos'è un Exploit e a cosa serve l'Address Space Layout Randomization?
Un Exploit è un codice che viene scritto per ottenere privilegi che non ti appartengono sfruttando una vulnerabilità. L'ASLR è la tecnica tramite la quale il sistema operativo, quando carica un programma da eseguire, per collocare i segmenti in memoria aggiunge una quantità casuale agli indirizzi base.

87. In linguaggio assembly, cos'è un'istruzione di salto e quali tipi di istruzione di salto possiamo trovare?
Un'istruzione di salto (JMP) va a modificare il registro CS (Code Segment) che indica l'istruzione successiva. Quindi l'istruzione che verrà eseguita dopo il JMP non sarà quella scritta dopo ma quella indicata dalla label specificata.
Possiamo avere istruzioni di salto:
- Condizionale: se la condizione è vera salta all'indirizzo specificato, altrimenti prosegue con l'istruzione che nell'eseguibile si trovo dopo.
- Non condizionale: senza alcuna condizione.
- Assoluto: ossia viene specificato di andare all'indirizzo x (JMP 1000, JNE ZZ).
- Relativo: viene indicato a quante istruzioni di distanza si trova l'istruzione dove bisogna saltare da quella attuale (JRE 250)

88. Spiegare cosa sono i processi "I/O bound" e quelle "CPU bound".
I processi "I/O bound" sono caratterizzante da un'alta richiesta di input/output, ossia impiegano più tempo a richiedere dati (attività di I/O burst) che ad elaborarli. Un processo, invece, "CPU bound" è caratterizzato da un altissimo consumo di CPU (attività di CPU burst) rispetto all'I/O.

89. Che cos'è una ready queue?
È la struttura dati in cui lo scheduler mantiene i PCB dei processi pronti ad essere eseguiti.

90. Spiegare la differenza tra "scheduling preemptive" e "scheduling non preemptive".
Le due situazioni dipendono da come è implementato lo scheduler. Uno scheduler di tipo preemptive nel momento in cui un processo entra in ready se ha una priorità maggiore rispetto a quello in esecuzione, quest'ultimo viene interrotto per far partire quello appena entrato. Uno scheduler di tipo non preemptive, invece, anche se il processo che entra in ready ha priorità maggiore esso dovrà aspettare la terminazione di quello in esecuzione.

91. Che cos'è il dispatcher?
È una parte del SO che passa effettivamente il controllo della CPU al processo selezionato dallo scheduler a breve termine.

92. Che cos'è il Throughput (produttività) e il tempo di turnaround?
Il throughput è il numero di processi completati nell'unità di tempo.
Il tempo di turnaruound (tempo di completamento) è il tempo necessario per eseguire un determinato processo.

93. Descrivere i vari algoritmi di scheduling.
FCFS/FIFO: quando un processo entra in coda viene inserito in fondo
SJF (Shortest Job First): seleziona il prossimo processo prelevando quello con i tempi di esecuzione minori (può generare starvation).
Priority Scheduling: ad ogni processo viene assegnato un numero che rappresenta la sua priorità, quindi verranno eseguiti prima i processi con priorità maggiore.
Round Robin: utilizza il metodo FCFS, però ogni volta che un processo viene messo in esecuzione lo scheduler imposta un timer, scaduto il quale il processo viene interrotto e reinserito nella ready queue per dare l'impressione di una esecuzione simultanea di tutti i processi.

94. Che cos'è il convoy effect?
È ciò che può avvenire in uno scheduling di tipo FCFS, ovvero che un processo molto lungo può bloccare molti processi che avrebbero eseguito molto più velocemente aumentando così i tempi medi di attesa.

95. È possibile adottare politiche differenti di scheduling?
Si, grazie alle code multiple. In ogni coda può essere implementata una politica di scheduling differente.

96. Come sono fatti i dischi meccanici (magnetici) ?
Sono composti da un insiemi di piatti. Ogni piatto è diviso in tracce circolari che a loro volta sono divise in settori. L'insieme delle tracce equidistanti dal centro si dice cilindro.
Per accedere ad un settore bisogna specificare la superficie, la traccia e il settore stesso.

97. Cos'è la formattazione fisica o formattazione a basso livello?
La formattazione fisica è essenzialmente la preparazione del disco, da vergine viene suddiviso con questo tipo di formattazione, in sezioni e per ogni sezione viene predisposta un'area per il controllo degli errori. Uno disco formattato a basso livello è utilizzabile solo dalla macchine e non dall'utente perché non è ancora stato definito un File System utilizzabile che verrà successivamente creato con una formattazione ad alto livello o logica.

98. Spiegare come avviene il bootstrap nei sistemi moderni.
Il bootstrap viene effettuato in due fasi.
Nella prima viene letto in piccolissimo programma all'interno della ROM, detto bootstrap loader, che avvia il boot che è il programma completo di avvio del Sistema Operativo che si trova nell'area di boot sul disco (RAM).

99. Perché non parliamo di politiche di scheduling del disco quando abbiamo a che fare con dischi a stato solido?
Perché a differenza dei dischi meccanici non ho parti in movimento perciò posso leggere in tempo costante in ogni punto del disco in un qualunque momento.

100. Elenca e descrivi i tipi di scheduling del disco?
- FCFS: la prima richiesta arrivata è la prima ad essere eseguita.

- SSTF (Shortest Seek Time First): vengono servite prima tutte le richieste più vicine alla posizione corrente. Soffre di Starvation, le richieste più periferiche rischiano di non essere mai soddisfatte.
- SCAN: il braccio si sposta da un estremo all'altro del disco servendo tutte le richieste che incontra.
- C-SCAN: opera come SCAN ma non inverte mai la direzione, ovvero una volta arrivato a fine corsa il braccio riparte dalla parte opposta (traccia 0).
- C-LOOK: come C-SCAN ma il bracci non apriva a fine corsa ma si ferma all'ultima richiesta, risparmiando tempo inutile per arrivare all'ultima traccia del disco.

101. Ipotizziamo che i cilindri di un disco magnetico siano indicizzati in maniera crescente dalla periferia verso il centro.
Supponiamo che la testina di lettura di un disco si trovi sul cilindro di indice 10. Ipotizziamo che la testina, per arrivare alla posizione attuale, si sia spostata verso il centro del disco.
Supponiamo che l'elenco delle richieste pendenti sia la seguente, ordinata in ordine di tempo di arrivo delle richieste.
12, 15, 6, 3, 6, 11, 13, 17, 20, 1.
Se uso un algoritmo di scheduling Shortest Seek Time First (SSTF), quale sara' la sequenza temporale dei prossimi accessi a disco?
Risposta: 11, 12, 13, 15, 17, 20, 6, 6, 3, 1
Accendo nella posizione più vicina a quella attuale (algoritmo greedy).

102. Nelle stesse ipotesi dell'esercizio precedente, quale sarà la sequenza temporale dei prossimi accessi a disco se invece uso una politica di scheduling del disco di tipo C-LOOK?
Risposta: 11, 12, 13, 15, 17, 20, 1, 3, 6, 6
Continuo nella direzione corrente fino a quando non incontro l'ultima richiesta in quella direzione (cioè che non provoca inversione) e vado alla richiesta più lontana nell'altro verso e torno indietro.

103. A cosa serve il programma ln?
È un programma che consente di creare collegamenti simbolici (in windows la frecciata sulle icone, se cambio ad esempio la posizione di un file/directory il collegamento non è più valido) o fisici (ciò che collega una directory al suo contenuto, se cambia il contenuto il collegamento rimane valido) a file e directory.

104. A cosa serve il programma fdisk?
È un'utility a linea di comando per la gestione dei dischi per creare e manipolare le tabelle delle partizioni e le partizioni stesse (ridimensionamento, modifica ed eliminazione).